UNDER REVIEW

RESCIENCE C

Replication/ASSIGNMENT

[Re] Replicating results of Neller and Presser [1] "Optimal Play of the Dice Game Pig"

Rich Cheng[1], James Marriner[1], Charlotte Rigby,[1] Maria-Louiza Van den Bergh[1]

[1] STOR-i, Lancaster University

Edited by

Reviewed by

Received
–

Published
–

DOI
–

1 Introduction

In the present article, we replicate the results of Neller and Presser [1] in their attempt to find the optimal policy for a player during the game of pig; they calculate this policy using a value iteration algorithm.

1.1 Game of Pig

The game of pig is a dice game, consisting of two players each of whom is aiming to be the first to reach 100. Both players start with a total score of 0. A single turn consists of the roll of a die. If the player rolls a 1, their turn comes to an end and their score for that turn, or the *turn total*, is set to 0; if the player rolls any other number, this number is added to their turn total and they are offered the choice to *roll* or *hold*. This is repeated until a player chooses to hold; once a player chooses to hold, their turn comes to an end and their turn total is added to their total score.

There are many different strategies players could use while playing pig. One particularly simple strategy which performs relatively well is "hold at 20"; for this, a player will continue to roll until their turn total is over 20, or until they roll a 1. Knizia [2] showed that when a player's turn total is less than 20 it is more beneficial to continue to roll until you reach 20. However, this may not always be the more effective strategy dependent on the state of the game. For example, if the total score of your opponent is 98 and your total score is 77, it may be worth using "riskier" behaviour and continue to roll even past 20 since there is a high likelihood that your opponent will reach 100 on their next turn.

2 Methodology

2.1 Problem Set Up

Neller and Presser [1] look to maximise the probability of winning. Let the player's total score be i, the opponent's total score be j and k be the player's turn total. Then player's probability of winning in that particular state is denoted $P_{i,j,k}$.
If $i + k \geq 100$,

$$P_{i,j,k} = 1,$$

Correspondence should be addressed to ()
The authors have declared that no competing interests exists.
Code is available at https://github.com/rescience-c/template.

since a player can hold and win.
If $0 \leq i, j < 100$ and $i + k < 100$ then:

$$P_{i,j,k} = \max(P_{i,j,k,roll}, P_{i,j,k,hold}) \tag{1}$$

with

- $P_{i,j,k,roll}$: the probability of winning if you were to roll;
- $P_{i,j,k,hold}$: the probability of winning if you were to hold.

These probabilities are given by:

$$\begin{aligned} P_{i,j,k,roll} &= \frac{1}{6}[(1 - P_{j,i,0}) + P_{j,i,k+2} + P_{j,i,k+3} + P_{j,i,k+4} + P_{j,i,k+5} + P_{j,i,k+6}] \\ P_{i,j,k,hold} &= 1 - P_{j,i+k,0} \end{aligned} \tag{2}$$

The probability of winning after either rolling a 1 or holding is the probability that your opponent will not win beginning with the next turn.

Therefore to compute the optimal policy for play, we need only solve for all probabilities of winning in all possible game states; then, we need only compare $P_{i,j,k,roll}$ and $P_{i,j,k,hold}$ for the current state and either roll or hold depending on which has the highest probability of success. Solving for the probabilities, however, can be complicated, since the dependencies between variables are cyclic. For example, $P_{i,j,0}$ depends on $P_{j,i,0}$ which in turn depends on $P_{i,j,0}$. This can be shown if players roll a 1 in subsequent turns, meaning that game states can repeat, and so we cannot just evaluate the probabilities from the final state back to the beginning.

2.2 Value Iteration

The game Pig can be modelled as a Markov Decision Process, with a state space $\mathcal{S}$ comprising of all feasible combinations of player score, opponent score and turn total, i.e. $s = \{i, j, k\}$ and an action space $A = \{\text{Roll, Hold}\}$.
Value Iteration is an iterative dynamic programming method used to estimate the value of each game state until convergence. To determine the optimal strategy, we apply the value iteration algorithm. Let $V(s)$ denote the estimated probability of eventually winning from state s. The update is given by:

$$V(s) \leftarrow max_a \sum_{s'} \mathcal{P}^a_{ss'}(\mathcal{R}^a_{ss'} + \gamma V(s'))$$

with:

- $\mathcal{P}^a_{ss'}$: probability of transitioning from state s to s' after taking action a;
- $\mathcal{R}^a_{ss'}$: immediate reward associated with the transition;
- γ : discount factor which determines the importance of future rewards.

Note that the maximisation is taken over all possible actions.

For Pig, terminal winning states, meaning the player can simply hold and win, are assigned value 1. All other rewards are 0. Following Neller and Presser [1], we use $\gamma = 1$, since we do care about future rewards given that the objective is to maximise the probability of eventually winning. The value estimates are then repeatedly updated until:

$$max_s |V^{(n)}(s) - V^{(n-1)}(s)| < \epsilon.$$

The whole method is summarised below in Algorithm 1.

Algorithm 1 Value Iteration

For each $s \in \mathcal{S}$, initialise $V(s)$ arbitrarily
repeat $\triangle \leftarrow 0$
  for each $s \in \mathcal{S}$, **do**
    $v \leftarrow V(s)$
    $V(s) \leftarrow \max_a \sum_{s'} \mathcal{P}^a_{ss'}(\mathcal{R}^a_{ss'} + \gamma V(s'))$
    $\triangle \leftarrow \max(\triangle, |v - V(s)|)$
  end for
until $\triangle < \epsilon$

2.3 Partitioned Value Iteration

To improve convergence speed, Neller and Presser [1] applied value iteration in stages by partitioning states according to the sum of the players' scores. The partitions were then solved in descending order of score sum.

The process begins by computing $P_{99,99,0}$ using value iteration. Once this value has converged, it is treated as fixed when computing the next partition, consisting of $P_{98,99,0}$ and $P_{99,98,0}$. The algorithm then proceeds iteratively through partitions with total score sums of 196, 195, and so on, until finally computing $P_{0,0,k}$ for $0 \leq k \leq 99$. In practice it proceeds as below.

Algorithm 2 Partitioned Value Iteration

for each $g = 0, 1, \ldots, 2 \cdot (\text{Target} - 1)$ **do**
  $\Delta \leftarrow \infty$
  $\text{partition}_g = \{0 \leq i \leq j \leq 99 : i + j = g\}$
  for each pair $(i^*, j^*) \in \text{partition}_g$ **do**
    Compute valid_states = $\{(i^*, j^*, k)$ and $(j^*, i^*, k) \in S, 0 \leq k \leq 100\}$
    repeat
      $v_{\text{old}} \leftarrow V$
      for each s in valid_states **do**
        $V(s) \leftarrow \max_{a \in A} \sum_{s'} \mathcal{P}^a_{ss'}(\mathcal{R}^a_{ss'} + \gamma \cdot v_{\text{old}}(s'))$
        $\Delta \leftarrow \max(\Delta, \mid v_{old}(s) - V(s) \mid)$
      end for
    until $\Delta < \epsilon$
  end for
end for

2.4 Implementation Details

Our reproduction was implemented in Python using packages NumPy and Matplotlib. The value iteration algorithm was implemented using an object-oriented structure with synchronous Jacobi updates. Since the exact implementation parameters were not specified by Neller and Presser [1], we selected reasonable parameter values for the reproduction. State values were initialised with $V(s) = 0$ for all states, convergence was determined using a tolerance of $\epsilon = 10^{-12}$ and we allowed a maximum of 100,000 iterations. This code only took roughly 15 minutes to run, allowing for us to have fairly strict criteria whilst maintaining reasonable computation time. The figures were generated directly from the converged optimal policy and reachable state space. Certain results involving playing policies against one another involve an element of randomness, and therefore, as such we have set the seed; this seed varies for different experiments and the exact seeds used can be found in our github package here.

3 Results

3.1 Win Probabilities

The reproduced optimal policy produced win probabilities closely matching those reported by Neller and Presser [1]. When both players followed the optimal policy, the first player won with probability 53.19%, with a 95% confidence interval of $[52.97, 53.41]$. This is comparable with the result of 53.06% from the original paper.

Against the "hold at 20" strategy, the reproduced policy also achieved similar performance to that reported by Neller and Presser [1]:

- When the optimal player acted first, the optimal player won with probability 57.42% (95% CI: $[57.20, 57.64]$), compared to 58.74% in the original paper;
- When the "hold at 20" player acted first, the optimal player won with probability 50.59% (95% CI: $[50.38, 50.81]$), compared to approximately 52.24% reported by Neller and Presser [1].

Whilst the results are not exactly the same, since there is an element of randomness when playing the game of Pig this is to be expected. Therefore, overall, the reproduced results demonstrate strong agreement with the original work and confirm the strategic advantage of the optimal policy over simpler approaches.

3.2 Piglet Convergence

Piglet is a simplified version of Pig played with a coin toss rather than a die; each turn a player repeatedly flips a coin until a tail is flipped or the player holds. In Neller and Presser [1], they first implement value iteration on a game of Piglet with a target score of 2 for simplicity; the results can be found below in Figure 1 which was our attempt to recreate Figure 2 from Neller and Presser [1].

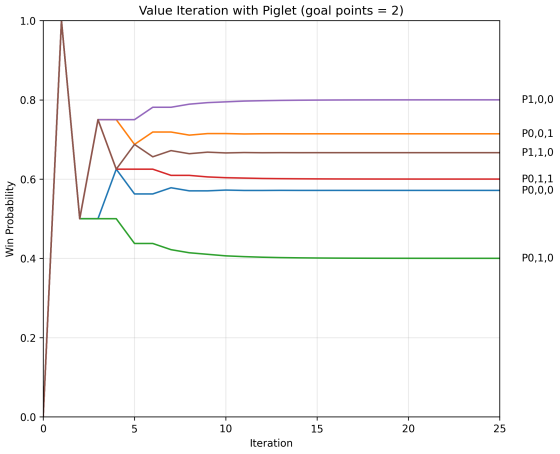

Figure 1. The convergence of state values for Piglet with a target score of 2, reproduced from Neller and Presser [1]

Note that the convergence trajectory differs slightly between our recreation and Neller and Presser [1]; however, our win probabilities converged to the same final state values. That is:

$$P_{0,0,0} = \frac{4}{7}, P_{0,0,1} = \frac{5}{7}, P_{0,1,0} = \frac{2}{5}, P_{0,1,1} = \frac{3}{5}, P_{1,0,0} = \frac{4}{5}, P_{1,1,0} = \frac{2}{3},$$

The difference in convergence rates may be due to the authors using asynchronous updates, rather than synchronous updates as we have. However, without their source code it is difficult to specify the exact reason for these discrepancies.

3.3 Optimal Policy Graphs

Figure 2 visualises the boundary between states where rolling and holding are optimal. The resulting surface demonstrates the highly non-linear structure of the optimal Pig policy. These appear to be highly similar to that of Figure 3 in Neller and Presser [1].

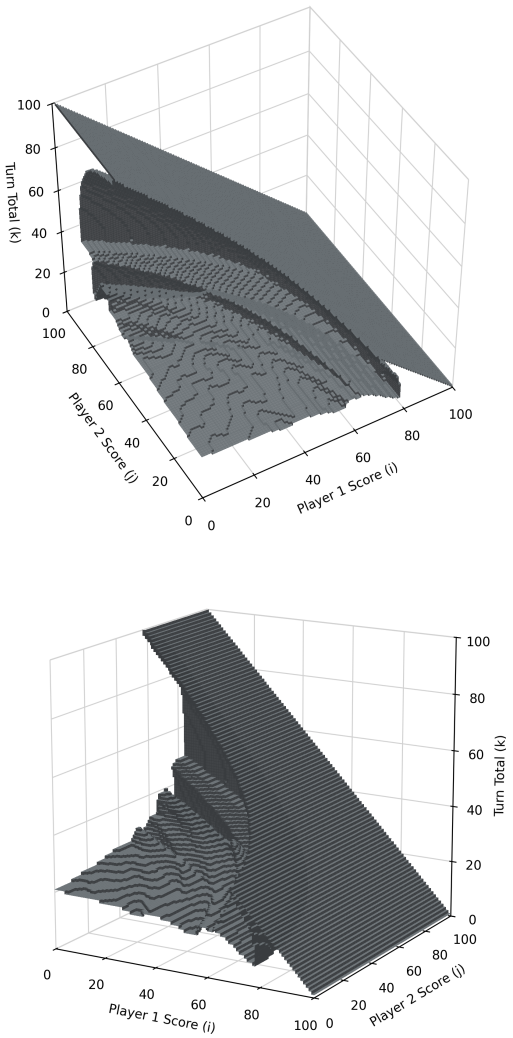

Figure 2. Visualisations of roll/hold boundary, reproduced from Neller and Presser [1]

We were also able to successfully recreate Figure 4 from Neller and Presser [1]. Shown in Figure 3 of our paper, it depicts a cross section of the reachable states if the opponent has a current score of 30. Also included is the cross section of reachable states with optimal play, along with a dotted line for comparison with the "hold at 20" policy.

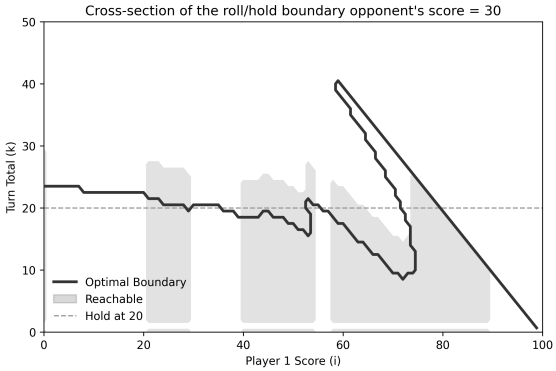

Figure 3. Cross section of the roll/hold boundary when opponent has score $j = 30$, reproduced from Neller and Presser [1].

3.4 Graph of Reachable States

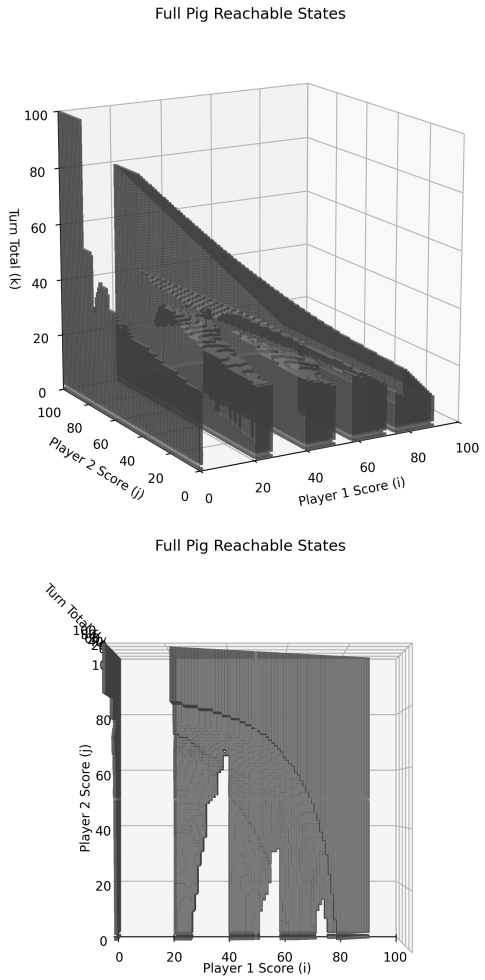

Figure 4. Visualisation of reachable states, reproduced from Neller and Presser [1].

Figure 4 is a recreation of Figure 5 in Neller and Presser [1], and illustrates the subset of game states which are reachable under optimal play. The gaps correspond to score combinations which are never encountered when following the optimal policy.

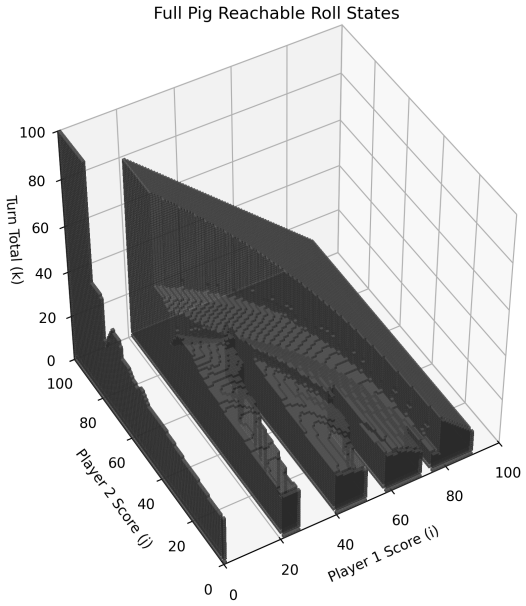

Figure 5. Reachable states where rolling is optimal, reproduced from Neller and Presser [1].

Figure 5 is a recreation of Figure 6 from Neller and Presser [1], and shows the reachable states for which rolling is the optimal action. As the player approaches the target score, the tendency to take risks increases.

3.5 Win Probability Contours

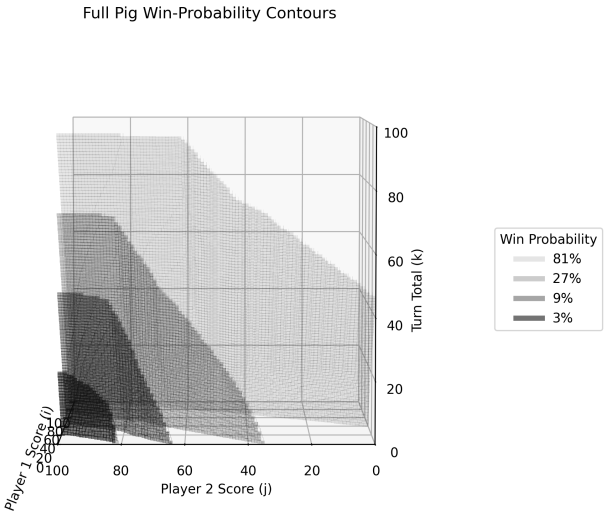

Figure 6. Win probability contours for optimal play $(3\%, 9\%, 27\%, 81\%)$, reproduced from Neller and Presser [1].

Finally, Figure 6 displays contours of equal win probability under optimal play. These contours illustrate how the likelihood of winning varies across different combinations of player score, opponent score and turn total. This very closely resembles the results found in the original paper.

4 Discussion

Although the majority of the numerical results and visualisations from Neller and Presser [1] were successfully reproduced to some degree, several challenges arose during the replication process. In particular, the original paper does not provide source code nor thorough implementation details; certain parameters such as the convergence threshold and initial $V(s)$ are not specified, which meant that certain aspects had to be inferred from the theoretical description alone and may not match up with the original implementation. These discrepancies meant that there were aspects of the report which were difficult to replicate.

4.1 The 5 R's

One of the standards we wish to compare both our own paper and the research paper against is the 5 R's. For the Neller and Presser [1], we have the following:

- Replicable
  The paper demonstrates a good level of replicability; the authors provide detailed mathematical derivations, state definitions and explanations of the value iteration algorithm which is used to compute the optimal Pig strategy.
- Re-Runnable
  The paper has limited re-runnability, since the original implementation details and source code are not provided.
- Repeatable
  Repeatability is partially supported given the detailed explanation of the methodology; however, as with re-runnability, given the lack of source code it is difficult to comment on the repeatability.
- Reproducible
  As with the previous cases, it is difficult to comment on the reproducibility without the source code. However, from a theoretical perspective given the thorough explanation, this could likely be reproduced in multiple different programming languages and achieve similar results. Furthermore, we were able to recreate the majority of the results.
- Reusable
  The methodology presented by Neller and Presser [1] demonstrates a strong degree of reusability. In particular, the authors discuss several possible extensions and variations of Pig to which the value iteration framework could be applied, illustrating that the approach is not restricted solely to the standard game formulation. However, without the source code, it is difficult to comment on the reusability of their code.

5 Summary

In this report, we reproduced the main computational results presented by Neller and Presser [1] concerning the optimal strategy for the game of Pig. Using value iteration

and partitioned value iteration, we successfully recreated the principal policy boundaries, state-space visualisations, and win probability estimates reported in the original work.

Overall, our results demonstrated strong agreement with those of Neller and Presser [1]. Whilst minor differences were observed in certain convergence trajectories and simulated win probabilities, the reproduced policies exhibited the same overall strategic behaviour and achieved comparable performance against heuristic strategies such as "hold at 20".

The reproduction process also highlighted the importance of detailed implementation information in computational research. Although the original paper provided a clear mathematical description of the methodology, the absence of source code and specific implementation parameters introduced challenges when attempting exact replication. Nevertheless, the work proved sufficiently well described to allow the successful independent reproduction of the majority of the reported results.

Finally, during our reproduction of Neller and Presser [1], we also attempted to stick to the 5 R's of reproducible research. We have provided a detailed description of the methodology, mathematical formulation and implementation choices. For reproducibility, we have included pseudocode for both value iteration and the partitioned value iteration used in Neller and Presser [1]. We have also tried to keep it as re-runnable and reusable as possible by including our source code, plotting scripts and instructions required for reproduction in the GitHub Repository here. However, our version of value iteration is slightly limited to the game of Pig and Piglet, limiting the reusability. Repeatability is supported here through the use of fixed random seeds where stochastic simulations were required, alongside explicitly stating the convergence parameters and implementation details.

References

1. T. W. Neller and C. G. M. Presser. "Optimal Play of the Dice Game Pig." In: **The UMAP Journal** (2004), pp. 25–47.
2. R. Knizia. **Dice Games Properly Explained**. Elliot Right-Way Books, 1999.